

Aula 13 — Controle embarcado na prática

Do liga-desliga ao PI, e onde entra IA

Microcontroladores — DENE/UFMT

OBJETIVOS

- Explicar por que uma saída de duas posições torna a oscilação inevitável, e o que muda quando a saída passa a ser fracionária.
- Derivar o erro em regime de um controlador proporcional a partir da potência necessária para manter o alvo.
- Reconhecer a saturação do integrador como consequência da planta, e não como refinamento, e implementar a proteção correspondente.
- Justificar, com o degrau do conversor, por que o termo derivativo não é viável nesta cadeia de medição.
- Descrever o fluxo de sintonia contra planta simulada, e as condições em que o simulador dá conselho errado.
- Situar honestamente o que é aprendido de máquina neste contexto: onde ele roda, o que desce para o microcontrolador e o que não desce.

Escopo, antes de tudo. Esta não é uma aula de teoria de controle. Não haverá transformação, lugar das raízes nem critério formal de estabilidade.

NOTA

O objetivo é outro: mostrar que **controle é feito em microcontrolador, e é assim que se faz**. O termostato desta bancada é o mesmo objeto que existe dentro de um ar-condicionado inverter, de uma estufa ou de uma incubadora. Tudo aqui é empírico e medido.

1 O teto do liga-desliga

O R9 mediu, nesta bancada, amplitude de 2,3 °C com histerese de 0,5 °C, e 16 acionamentos por hora.

Desses 2,3 °C, apenas 1,0 é a faixa de histerese. O restante é sobressinal, e o sobressinal não depende de h : vem da inércia térmica e do atraso entre aquecedor e sensor.

CONCEITO

Reduzir h a zero deixaria a amplitude em 1,3 °C e faria os acionamentos tenderem ao infinito. As duas grandezas pioram juntas, e não há ajuste que escape.

A causa está antes do algoritmo. Uma saída que só vale 0% ou 100% empurra a planta ao máximo numa direção, sempre. Ela nunca aplica **exatamente** a potência que compensa a perda de calor — porque não sabe aplicar valores intermediários.

2 De duas posições para uma fração

A mudança que torna tudo o resto possível não é de software: é de acionamento. O relé sai, entra o estágio eletrônico do encontro 8, comandado pelo PWM do mesmo encontro.

CONCEITO

A saída deixa de ser um bit e passa a ser um número de 0 a 1023. E, com isso, passa a existir um valor de saída em que a temperatura simplesmente **fica parada** no alvo — a potência entregue igual à perda.

O liga-desliga não tinha esse valor disponível. Nenhum algoritmo poderia encontrá-lo.

Duas consequências imediatas, e as duas são boas.

NOTA

O relé não comuta mais, então a conta de vida útil do encontro 9 desaparece do problema — o MOSFET não tem contato mecânico.

E a frequência de comutação passa a ser a do PWM, na casa do quilohertz, muito acima de qualquer coisa que a inércia térmica perceba. A planta vê apenas a média, como o encontro 8 estabeleceu.

3 Proporcional

A regra mais simples com saída fracionária: quanto mais longe do alvo, mais potência.

$$u = K_p \cdot e, \text{ com } e = \text{alvo} - T$$

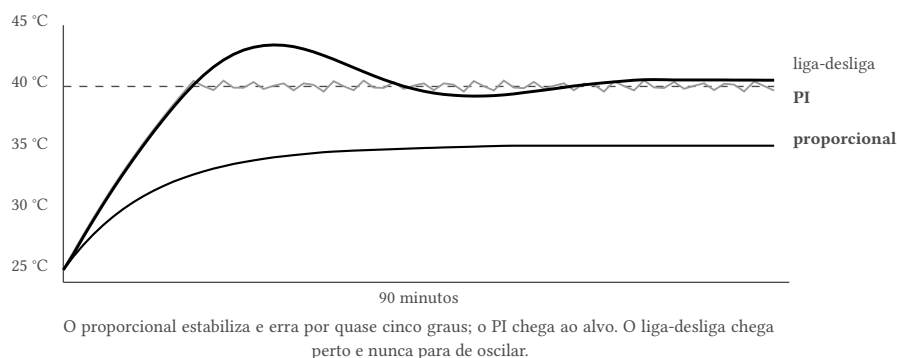


Figura 1: As três estratégias na mesma planta, partindo da temperatura ambiente. O proporcional estabiliza e para longe do alvo.

O erro em regime tem uma fórmula, e ela é desconfortável.

CONCEITO

Em equilíbrio a temperatura não muda, logo a potência entregue iguala a perdida. Se manter 40 °C exige 37,5% de potência, então em equilíbrio $K_p \cdot e = 0,375$.

Com $K_p = 0,05$ por grau, isso exige $e = 7,5$ °C. O controlador **precisa** de erro para produzir saída: erro zero significa saída zero, e saída zero significa a temperatura caindo.

O ponto de equilíbrio real resolve as duas relações ao mesmo tempo e cai em 35 °C — cinco graus abaixo do alvo, exatamente como a figura mostra.

A tentação é aumentar K_p . Isso funciona até certo ponto — o erro cai proporcionalmente — e depois deixa de funcionar.

ATENÇÃO

Com ganho alto, o atraso entre aquecer e medir faz o controlador reagir a uma informação velha. Ele continua empurrando quando já deveria ter parado, a temperatura passa do alvo, ele puxa demais, e a oscilação volta. Só que agora sem histerese e sem período previsível.

Erro em regime pequeno e oscilação são as duas pontas do mesmo ajuste. É por isso que existe o termo seguinte.

4 Integral

O termo integral acumula o erro ao longo do tempo:

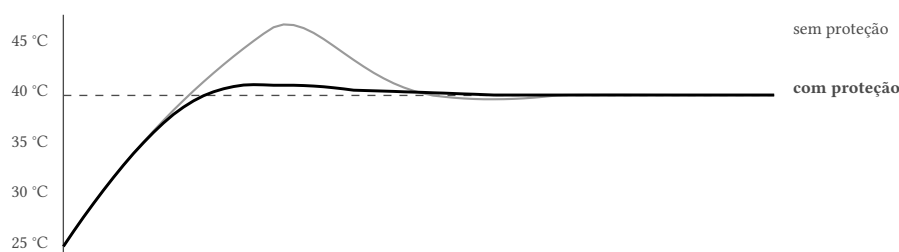
CONCEITO

$$u = K_p \cdot e + K_i \cdot \sum e \cdot T_s$$

Enquanto houver erro, a soma cresce e a saída sobe. A única situação em que a soma para de crescer é $e = 0$ — ou seja, o alvo exato.

Ele não substitui o proporcional: o proporcional reage rápido ao que está acontecendo agora, o integral corrige devagar o que sobrou.

4.1 A saturação que o integrador não vê



Durante o aquecimento a saída fica saturada em 100% por quarenta minutos. Sem proteção, o acumulador cresce esse tempo todo e cobra sete graus de sobressinal para se desfazer.

Figura 2: O mesmo controlador, com e sem proteção contra saturação. A diferença é de seis graus de sobressinal.

Partindo de 25 °C para 40, o erro inicial é de 15 °C. Com qualquer ganho razoável, a saída calculada passa de 100% e é limitada — o aquecedor já está no máximo e não há mais o que fazer.

CONCEITO

O integrador, porém, não sabe disso. Ele continua somando erro durante os **quarenta minutos** de aquecimento, e acumula um valor enorme que não teve efeito nenhum, porque a saída estava presa no teto.

Quando o alvo é finalmente alcançado, esse acúmulo ainda está lá, mandando aquecer. A temperatura passa do alvo e continua subindo até que o erro negativo consuma tudo que foi somado. No exemplo da figura, sete graus de sobressinal.

A proteção é uma linha: **não acumule quando a saída está saturada.**

ATENÇÃO

```
u = Kp * e + Ki * integral;
if (u > 0 && u < MAX) {           /* so integra na regio util */
    integral += e;
}
```

Nesta planta a saturação não é excepcional: ela é o estado normal durante todo o aquecimento. Por isso a proteção não é refinamento — é parte do controlador, e um PI sem ela simplesmente não funciona aqui.

5 Por que o derivativo não entra

O terceiro termo clássico reage à **velocidade** do erro, antecipando. Nesta cadeia de medição ele é inviável, e o motivo é um número que o curso conhece desde o encontro 4.

O degrau do conversor vale 0,488 °C. Perto do alvo, a temperatura muda a cerca de 0,01 °C por segundo.

CONCEITO

Amostrando a 1 Hz, a diferença entre duas leituras consecutivas é de 0,01 °C — cinquenta vezes menor que um degrau. O conversor devolve o **mesmo código** por quarenta e nove amostras seguidas, e na quinquagésima pula 0,488 de uma vez.

A derivada calculada é, portanto, zero quarenta e nove vezes e 0,488 uma vez. Ela não mede a inclinação: mede o instante em que o conversor trocou de degrau. É um trem de impulsos, e multiplicá-lo por um ganho injeta ruído na saída.

PI é a resposta honesta aqui, e dizer isso é mais útil que ensinar um termo que o aluno desligaria na primeira tentativa.

NOTA

O que tornaria o derivativo viável seria melhorar a **medida**: amostrar mais devagar para acumular variação real, filtrar, ou aumentar a resolução efetiva. Todas as três voltam ao encontro 4, e a última é a razão de o ADS1115 existir.

6 A autoridade é de um lado só

O aquecedor injeta energia. O resfriamento é passivo, somado à ventoinha.

CONCEITO

Acima do alvo, a única ação disponível é $u = 0$ e esperar — e a taxa de queda não é escolha do controlador, é propriedade da planta. Abaixo do alvo, há autoridade plena.

O controlador é **assimétrico por construção**, e um ganho único serve mal aos dois sentidos: o que é adequado subindo é agressivo demais descendo, onde de qualquer forma ele não tem o que fazer.

Isso não invalida o PI. Mas explica por que a sintonia perfeita não existe com um par de ganhos só, e é o ponto de entrada natural para o que vem a seguir.

7 Ponto fixo

O compilador na versão gratuita não tem ponto flutuante (encontro 4). Os ganhos descem como inteiros escalados.

```
/* Ganhos em Q8, em contagens de saída (0..1023) por decimo de grau:
   valor real = KP_Q8 / 256. Kp = 0,05 da saída por grau
   = 0,05 * 1023 / 10 = 5,1 contagens por decimo -> 5,1 * 256 = 1309. */
#define KP_Q8    1309
#define KI_Q8     4          /* por decimo e por segundo: o R13 sintoniza */

static int32_t integral = 0;

uint16_t controlar_pi(int16_t erro_d)    /* erro em decimos de grau */
{
    int32_t u = ((int32_t) KP_Q8 * erro_d + (int32_t) KI_Q8 * integral) >> 8;

    if (u > 1023) { u = 1023; }
    else if (u < 0) { u = 0; }
    else { integral += erro_d; }          /* so integra fora da saturacao */

    return (uint16_t) u;
}
```

Todos os produtos intermediários são de 32 bits, e não por precaução: o acumulador cresce mesmo. Dez minutos de erro de 5 °C amostrado a 1 Hz somam 30000 décimos de grau, que já não caberiam em 16 bits com sinal.

ATENÇÃO

É o mesmo defeito do encontro 4 com outro nome, e continua silencioso: o número dá a volta e o controlador passa a empurrar para o lado errado.

8 Sintonizar sem gastar a bancada

A planta tem constante de tempo de dezenas de minutos. Uma sessão de laboratório comporta duas ou três tentativas de sintonia, e uma sintonia decente precisa de dezenas.

O fluxo do R13, em três etapas.

CONCEITO

Identificar, uma vez, no hardware. Aquecedor a 100% a partir do ambiente, telemetria ligada, dez minutos. Desse registro saem a taxa de aquecimento, a constante de tempo e o atraso.

Sintonizar contra a planta simulada, em Python. Com o modelo ajustado, dezenas de combinações de ganhos rodam em segundos, e dá para varrer o espaço inteiro em vez de chutar duas vezes.

Confirmar na bancada, só os vencedores. Duas ou três combinações, medidas de verdade, com a mesma telemetria do R10.

É o mesmo fio do simulador que atravessa o curso desde o encontro 0, agora usado para o que ele faz de melhor: repetir barato.

O simulador mente, e desta vez a mentira é perigosa.

DIVERGÊNCIA

Uma planta simulada em ponto flutuante não tem o degrau de 0,488 °C, não tem ruído e não tem atraso de amostragem. Sintonizado contra ela, um controlador com termo derivativo parece excelente — e na bancada ele é ruído puro, pelo motivo da seção 6.

Coloque a quantização no simulador. Uma linha, arredondando a temperatura medida para o múltiplo de 0,488 mais próximo, e o modelo passa a recomendar o que a bancada aceita.

Saber onde o simulador mente é conteúdo desde o encontro 7. Aqui ele custaria caro.

9 Onde entra IA

O enquadramento honesto. O PIC18F4550 não treina nada, e quase não infere. O que ele faz é **executar o que foi aprendido**.

CONCEITO

E essa separação — treino fora, execução dentro — não é limitação didática: é o padrão industrial. É exatamente o que existe num ar-condicionado inverter ou numa geladeira com compressor de velocidade variável. O modelo é ajustado no laboratório do fabricante; o que vai para o produto são coeficientes.

O fluxo fecha com o resto do curso sem inventar nada:

De onde vêm os dados	Telemetria do R10 — horas de temperatura, saída e alvo
Onde roda o ajuste	Python, no computador, fora do microcontrolador
O que desce para o chip	Coefficientes em ponto fixo, uma tabela, ou regras
Onde ficam guardados	Memória não volátil do R11

9.1 O caso concreto: ganhos por região

A seção 7 deixou um problema: um par de ganhos serve mal a uma planta assimétrica.

A resposta clássica em controle embarcado é uma **tabela**.

Em vez de um K_p , uma pequena tabela indexada pela região de operação — longe abaixo do alvo, perto, acima. Os valores da tabela são ajustados em Python contra o modelo identificado, e descem como inteiros.

CONCEITO

O controlador no microcontrolador continua tendo vinte linhas. O que mudou é que os números dentro dele foram **aprendidos a partir de dados medidos**, em vez de escolhidos à mão.

Isso é aprendizado supervisionado no sentido mais literal e menos glamoroso do termo, e é o que efetivamente roda em produto. Historicamente, o passo seguinte neste domínio é um controlador **fuzzy** — regras com transição suave entre regiões —, que é o que mais se usa em controle de temperatura embarcado.

9.2 Uma rede que cabe em 26 bytes

Esta subseção inteira pode ser pulada. Ela existe para responder concretamente “como seria”, e o resultado dela é uma conclusão negativa útil.

OPCIONAL

A tabela da subseção anterior é o que se usa. Mas vale ver, com todos os números na mesa, o que significa **rodar uma rede** neste processador — porque o assunto costuma ser discutido sem que ninguém diga qual é o tamanho das matrizes.

A arquitetura. Duas entradas, três neurônios ocultos com retificação, uma saída linear. A tarefa é aprender a **potência de regime**: dada a temperatura atual e o alvo, qual saída mantém a planta parada ali.

CONCEITO

Isso é uma antecipação — em inglês, *feedforward* — e vai **somada** ao PI, não no lugar dele. O PI continua corrigindo o que a rede errar.

Matriz	Forma	Elementos	O que é
W_1	3×2	6	Pesos da camada oculta
b_1	3	3	Polarizações da camada oculta
W_2	1×3	3	Pesos da saída
b_2	1	1	Polarização da saída

Treze parâmetros. Em *int16*, vinte e seis bytes — cabem folgadoamente na memória não volátil do encontro 11, ao lado dos ganhos do PI.

As entradas precisam ser normalizadas antes de entrar, e essa é a parte que se esquece. Uma temperatura de 400 décimos e um erro de 15 décimos têm escalas diferentes por um fator de trinta; sem normalizar, o treino gasta a capacidade da rede aprendendo a escala em vez da relação.

NOTA

Aqui: $x_1 = (T - 25)/25$ e $x_2 = e/10$, ambos em ponto fixo Q8.

9.3 A inferência, em ponto fixo

Todos os pesos são inteiros em Q8 — valor real igual ao inteiro dividido por 256. O produto de dois números Q8 é Q16, e volta a Q8 com um deslocamento de oito.

```
#define NE 2                /* entradas          */
#define NO 3                /* neuronios ocultos */

/* Pesos em Q8, vindos da memoria nao volatil (encontro 11). */
static int16_t W1[NO][NE]; /* 3 x 2 */
static int16_t b1[NO];     /* 3      */
static int16_t W2[NO];     /* 1 x 3 */
static int16_t b2;         /* 1      */

static int16_t satura(int32_t v)
{
    if (v > 32767) { return 32767; }
    if (v < -32768) { return -32768; }
    return (int16_t) v;
}

/* x[] em Q8; devolve a saida em Q8. */
static int16_t inferir(const int16_t x[NE])
{
    int16_t h[NO];

    for (uint8_t j = 0; j < NO; j++) {
        int32_t acc = (int32_t) b1[j] << 8; /* polarizacao em Q16 */

        for (uint8_t i = 0; i < NE; i++) {
            acc += (int32_t) W1[j][i] * x[i]; /* Q8 * Q8 = Q16 */
        }
    }
}
```

```

    acc >>= 8;                                /* de volta para Q8 */
    h[j] = (acc > 0) ? satura(acc) : 0;        /* ativacao: max(0, .) */
}

int32_t saida = (int32_t) b2 << 8;
for (uint8_t j = 0; j < NO; j++) {
    saida += (int32_t) W2[j] * h[j];
}
return satura(saida >> 8);                    /* saida linear */
}

```

E o uso, somando a antecipação ao PI:

```

uint16_t controlar(int16_t t_d, int16_t alvo_d)
{
    int16_t x[NE];
    x[0] = (int16_t)((int32_t)(t_d - 250) * 256) / 250; /* (T-25)/25 */
    x[1] = (int16_t)((int32_t)(alvo_d - t_d) * 256) / 100; /* e/10 */

    int32_t ff = ((int32_t) inferir(x) * 1023) >> 8; /* Q8 -> 0..1023 */
    int32_t u = ff + controlar_pi(alvo_d - t_d);

    if (u > 1023) { u = 1023; }
    if (u < 0) { u = 0; }
    return (uint16_t) u;
}

```

A escolha da ativação não é estética. A retificação — $\max(0, x)$ — é uma comparação e uma atribuição. A tangente hiperbólica exigiria uma tabela de trinta e duas entradas com interpolação, ou uma aproximação polinomial: dezenas de ciclos por neurônio, mais Flash.

CONCEITO

Numa rede de três neurônios a diferença de qualidade é desprezível e a diferença de custo não é. Retificação, aqui, não é simplificação didática: é a escolha certa.

O custo, em ciclos. São nove multiplicações-acumulações de 16 bits, mais as normalizações e os deslocamentos. Da ordem de **500 ciclos**, ou 125 μs .

ATENÇÃO

Compare com a tabela do curso: 60 ciclos para uma conversão, 5 200 para atualizar a tela. A inferência custa cerca de um décimo de uma escrita no display, uma vez por segundo.

Meça o seu número com um pino e o osciloscópio, em vez de acreditar nesta estimativa — a multiplicação de 16 bits no PIC18 é feita em partes, e o compilador tem liberdade sobre como.

A conclusão que importa: **a inferência cabe com folga**. O que não cabe é o treino.

9.4 Quantizar: escolher onde fica a vírgula

Isto não é um contorno. A tentação é achar que a ausência de ponto flutuante nos obriga a inventar algo caseiro. É o contrário: **quantização inteira é o caminho padrão** de qualquer implantação em borda. As ferramentas de mercado convertem modelos treinados em ponto flutuante para inteiros antes de embarcá-los, e por motivos que não têm nada a ver com este chip — inteiros são mais rápidos, ocupam menos e gastam menos energia em qualquer plataforma.

NOTA

O que fazemos aqui à mão é o que essas ferramentas fazem automaticamente. Vale conhecer os nomes: **quantização pós-treino, escala e ponto zero, requantização entre camadas, treino consciente de quantização**.

Escolher o formato é escolher entre faixa e resolução. Num `int16`, os dezesseis bits são repartidos entre parte inteira e fracionária, e a posição da vírgula é uma decisão de projeto:

CONCEITO

Formato	Faixa	Resolução	Serve para
Q4	± 2048	0,0625	Acumuladores, somas longas
Q8	± 128	0,0039	Pesos e ativações desta rede
Q12	± 8	0,00024	Pesos pequenos, sem folga de faixa

Se um peso treinado valer 200, ele não cabe em Q8 — e o resultado não é um erro de compilação, é um número que dá a volta. A verificação obrigatória depois do treino é olhar o maior peso em módulo e confirmar que ele cabe no formato escolhido.

É a terceira vez que este curso encontra exatamente este compromisso: PR2 contra resolução do PWM no encontro 8, histerese contra acionamentos no encontro 9, e agora faixa contra resolução na vírgula. A forma é sempre a mesma.

Onde a precisão realmente se perde. Há três lugares, e eles não têm o mesmo peso.

ATENÇÃO

Nos pesos: arredondar para Q8 introduz erro de até 1/512 em cada um. É pequeno e é estático — dá para medir uma vez e esquecer.

Nas ativações: o acumulador tem 32 bits, mas `h[j]` volta para 16. Essa requantização entre camadas é onde a perda se acumula em redes profundas. Com uma camada oculta, é desprezível.

Na entrada: e aqui está o número que encerra a discussão. O conversor entrega degraus de 0,488 °C. Normalizada por 10 e escalada por 256, uma única unidade de código do conversor equivale a **doze contagens** em Q8.

Ou seja, a entrada chega ao modelo com uma granularidade doze vezes mais grossa do que a aritmética interna consegue representar. Refinar os pesos para Q12 não compraria nada: o gargalo está antes, no encontro 4.

Por que `int16` e não `int8`. O padrão de mercado é `int8`, e por um bom motivo: metade da memória e multiplicação mais barata.

NOTA

Neste chip a vantagem seria ainda maior. O PIC18 tem multiplicador de 8 por 8 bits em **uma instrução**; uma multiplicação de 16 por 16 é feita em quatro produtos parciais mais somas. Passar a rede para `int8` reduziria a inferência a algo como um terço dos ciclos.

Usei `int16` aqui por folga de faixa e clareza. Uma rede em `int8` bem escalada é mais rápida, mais padrão e mais trabalhosa de acertar — e é exatamente o exercício que sobra para quem quiser ir adiante.

Saturar, nunca dar a volta. A função `satura` do código anterior não é zelo excessivo.

ATENÇÃO

Se um acumulador estourar e der a volta, um valor positivo grande vira um negativo grande, e o controlador passa a empurrar o aquecedor exatamente para o lado errado, com autoridade máxima. Saturando, o pior caso é a saída ficar presa no extremo — errada, mas na direção certa.

É o mesmo raciocínio do limitador de saída do PI, e o mesmo defeito silencioso do encontro 4. Terceira aparição.

9.5 O que acontece do lado do computador

Etapas	Forma dos dados	Observação
Registro bruto do R10	$N \times 5$, com $N \approx 28800$	Oito horas a 1 Hz
Rotulagem	$X: N \times 2, Y: N \times 1$	Só as amostras com temperatura estável entram
Treino	13 parâmetros	Minutos, em Python

Escolha de formato	—	Olhar o maior peso e decidir a posição da vírgula
Quantização	13 inteiros int16	Multiplicar por 256 e arredondar
Verificação	—	Resimular com os pesos arredondados

Duas etapas dessa tabela costumam ser esquecidas, e as duas dão trabalho.

CONCEITO

Rotular não é gravar. O registro do R10 tem temperatura e saída a cada segundo, mas o que a rede precisa aprender é a saída **de regime** — aquela aplicada quando a temperatura já não muda. As amostras colhidas durante o aquecimento têm saída máxima e temperatura qualquer, e ensinariam exatamente a coisa errada. Filtrar por $|dT/dt| \approx 0$ é o passo que transforma dado em rótulo.

Quantizar é uma segunda modelagem. Os pesos treinados em ponto flutuante viram inteiros por arredondamento, e o comportamento muda um pouco. A verificação obrigatória é resimular a planta com os pesos já arredondados, antes de gravá-los. Um modelo validado em ponto flutuante não é o modelo que vai rodar.

9.6 O resultado honesto

Treine essa rede com os dados desta bancada e compare com uma reta ajustada aos mesmos pontos — dois parâmetros em vez de treze.

ATENÇÃO

A reta ganha. Nesta planta, nesta faixa de temperatura, a potência de regime é proporcional à diferença entre a temperatura e o ambiente. A relação é linear, e treze parâmetros não descrevem uma reta melhor que dois.

A rede custa 500 ciclos contra algumas dezenas, 26 bytes contra 4, e um procedimento de treino contra uma fórmula fechada.

E quando ela passaria a ganhar? Quando a relação deixasse de ser uma reta:

CONCEITO

com a ventoinha em velocidade variável, a perda passa a depender também da saída dela, e a superfície ganha curvatura;

em temperaturas bem mais altas, a perda por radiação cresce com a quarta potência e a linearidade se rompe;

se o ambiente variar bastante, a entrada passa a ser tridimensional e a interpolação manual fica desagradável.

Nenhuma dessas condições vale nesta bancada — e é exatamente por isso que o exercício vale a pena. O aluno sai com a pergunta certa, que não é **como uso uma rede**, e sim **o que na minha planta justifica uma**.

O risco a evitar. Se esta seção virar vinte minutos de transparências sobre redes neurais, é ruído.

ATENÇÃO

Ela só se justifica se o aluno **computar** um coeficiente a partir dos dados que ele mesmo coletou, gravá-lo na memória, e ver o comportamento da planta mudar. Caso contrário, corte a seção — o curso não perde nada.

10 Previsão para o R13

PREVISÃO

P1. Do registro do degrau, estime a taxa de aquecimento inicial e a constante de tempo. Escreva as duas antes de ajustar qualquer modelo.

P2. Com K_p sozinho e o valor que você escolher, em que temperatura espera que o sistema estabilize? Use a conta da seção 4.

P3. Ligue o integral sem proteção contra saturação e preveja o sobressinal em graus. Depois meça.

P4. Quantas amostras seguidas você espera ler o **mesmo** código do conversor com o sistema estabilizado no alvo? O que isso diz sobre derivar essa medida?

P5. Compare a amplitude e o número de acionamentos do PI com os números que o R9 mediu para o liga-desliga. Qual das duas grandezas melhorou mais?

Leve esta folha e o `degrau.csv` do R10. Cada pergunta vale 0,4 — 2,0 no total do R13. As Tarefas 1 e 2 do R13 são feitas antes da sessão, no notebook do Colab: a bancada é para confirmar, não para descobrir.

SEM NOTA

11 Exercícios

Exercício 13.1. Manter 45 °C nesta planta exige 55% de potência. Um controlador proporcional usa $K_p = 0,08$ por grau.

TAREFA

(a) Qual erro em regime a fórmula simples prevê?

(b) Por que a temperatura de equilíbrio real é mais alta que a prevista por essa conta?

(c) Dobrar K_p resolve?

Exercício 13.2. Um colega implementa o PI e observa que, ao ligar o sistema frio, a temperatura sobe até 8 °C acima do alvo antes de voltar.

TAREFA

(a) Qual é o defeito?

(b) Por que ele não aparece quando o alvo é alterado de 40 para 41 °C com o sistema já quente?

(c) Corrija.

Exercício 13.3. Explique, com números, por que o termo derivativo não é viável nesta cadeia de medição, e cite duas mudanças que o tornariam viável.

TAREFA

Exercício 13.4. Um colega sintoniza os ganhos contra a planta simulada em Python e obtém excelente desempenho com termo derivativo. Na bancada, a saída tremula e o aquecedor pisca.

TAREFA

(a) O que o simulador não estava representando?

(b) Como corrigir o simulador?

(c) Que princípio geral este episódio ilustra?

Exercício 13.5. Depois do treino, o maior peso em módulo da rede vale 3,7 e o menor não nulo vale 0,012.

TAREFA

(a) O formato Q8 em int16 acomoda os dois? Mostre a conta.

(b) E Q12? E Q4?

(c) Um colega propõe passar tudo para int8 em Q4. O que ele ganha e o que perde?

(d) Vale a pena refinar o formato dos pesos para reduzir o erro do modelo nesta bancada? Justifique com o degrau do conversor.

Nos encontros 14 e 15: seminários comparativos.

NOTA

A ponte sai de dentro desta aula. O termo derivativo, o filtro que o viabilizaria e qualquer inferência mais pesada são justamente o que este processador não roda bem — e são a

primeira coisa que uma plataforma de 32 bits com ponto flutuante em hardware resolve sem esforço.

A pergunta do seminário não é qual plataforma é melhor. É **o que exatamente muda**, e quanto disso importa para um termostato.